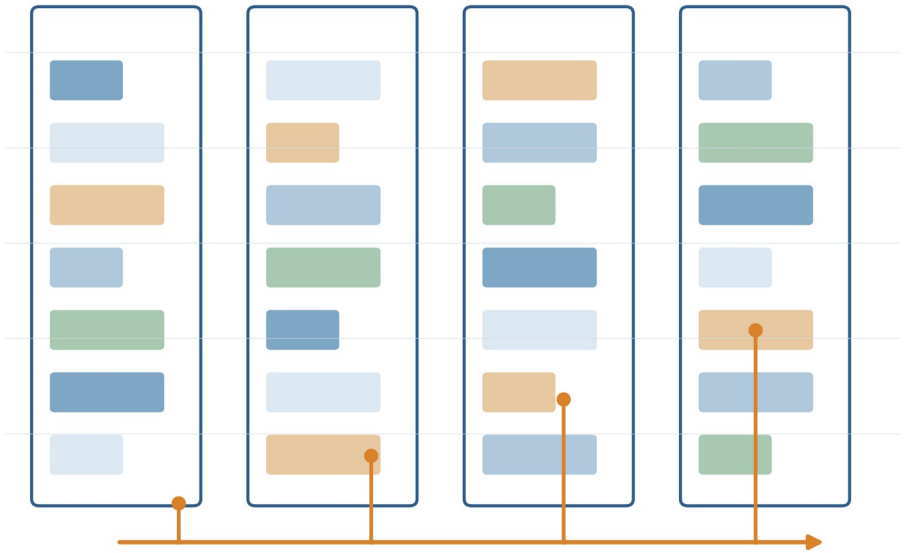
Revisiting Lifetime-Aware VM Allocation

Does better prediction make better VM placement?

Part 1 The problem
Part 2 What we tested
Part 3 Our improvement

CE7490 Course Project · 26S1 Group 5

Zhang Qi (G2510784F)
qi015@e.ntu.edu.sg
Zhang Saining (G2511061H)
saining002@e.ntu.edu.sg
Zheng Yitao (G2608664B)
YZHENG056@e.ntu.edu.sg



Based on LAVA (MLSys 2025)
Synthetic reconstruction with paired tests

VM placement uses both capacity and time

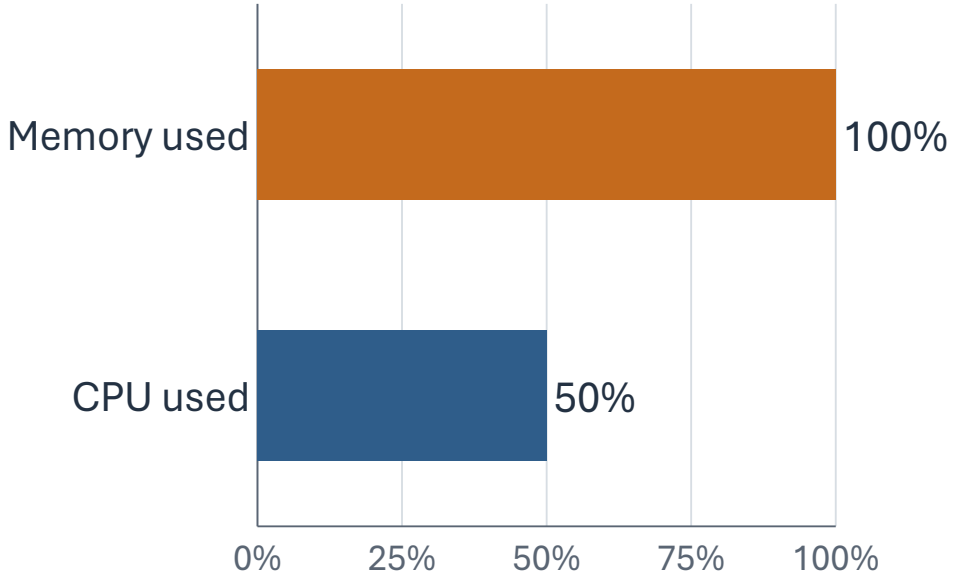
A placement must fit CPU and memory. Departures determine when a whole host becomes empty.

VM: a virtual computer

Host: a physical server running several VMs

A request fits only when BOTH resources are available.

Even one remaining VM prevents the host from becoming completely empty.



Capacity example: CPU has room, memory is full.

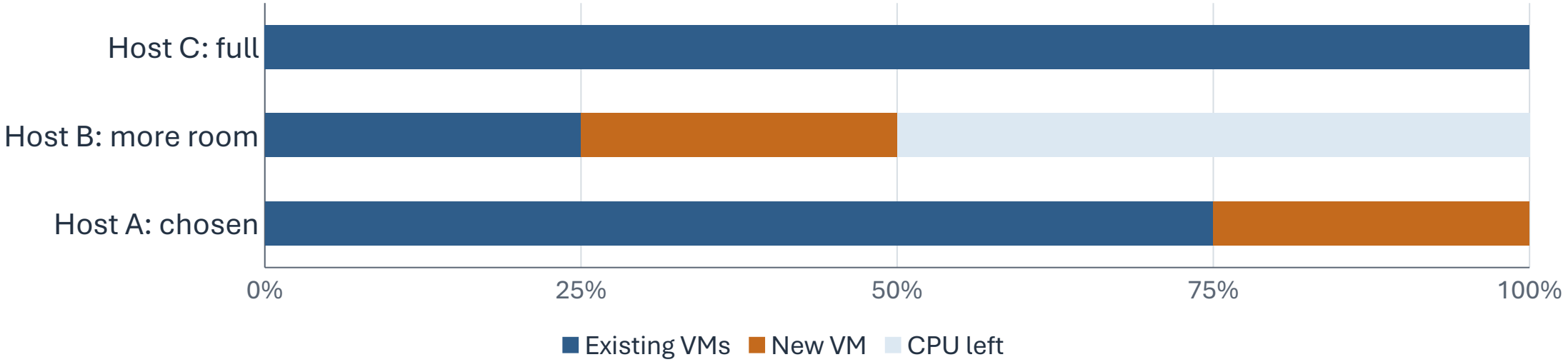
Our metric measures time with empty hosts. It does not directly measure electricity savings.

Best-Fit chooses the tightest feasible host

Best-Fit is the baseline: use the current resource space to select a feasible placement.

1 Check CPU and memory limits

2 Prefer less unused space after placement



Example VM: 25% CPU and 12.5% memory. All feasible candidates have enough memory.

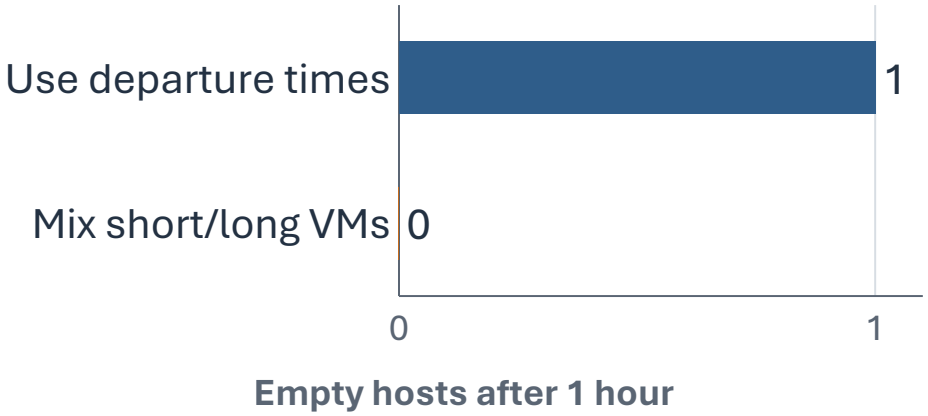
Best-Fit uses current space. LAVA additionally uses expected departure times.

LAVA adds expected departure times

The same VMs can occupy hosts for different lengths of time, depending on how they are grouped.

Four simultaneous arrivals: two run for 1 h, two for 10 h. Each host fits two VMs.

Arrangement	Host A	Host B
Mixed	1 h + 10 h	1 h + 10 h
Similar exits	1 h + 1 h	10 h + 10 h



After 1 hour: grouping similar departures frees one whole host.

This illustrates the intuition. Full LAVA also updates predictions and adapts host lifecycle states.

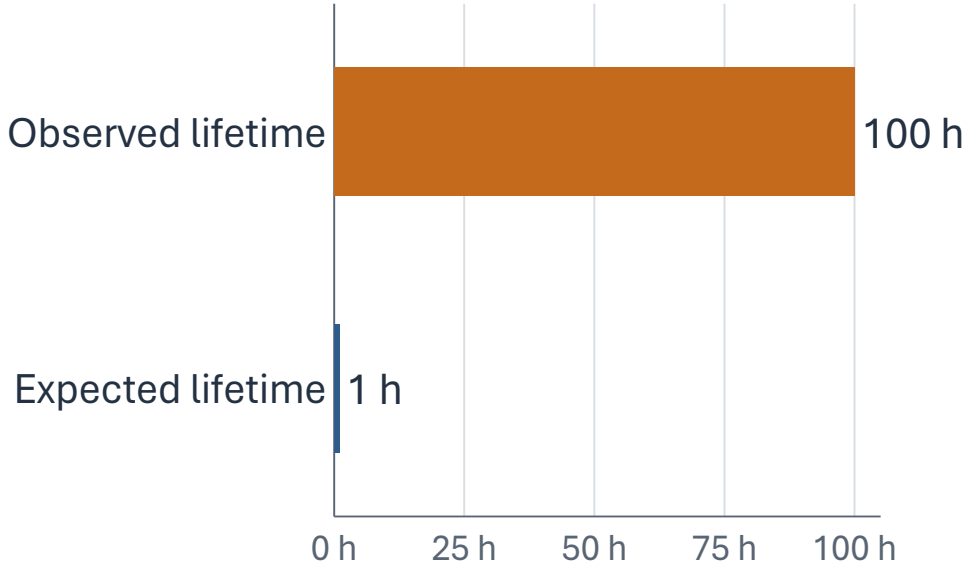
Prediction errors can reverse the benefit

An unexpectedly long VM can keep a host busy after the other VMs leave.

Illustrative prediction error

A VM looks short, so the scheduler expects its host to empty soon.

If it runs much longer, that planned departure alignment breaks.



LAVA adapts to mispredictions. We test how much the initial prediction quality still matters.

A paired experiment separates method from workload

Every policy receives the same arrivals, resource demands and noisy observations for each seed.

Design	Setting	Why it matters
Workload	180 hosts, 2,500 VMs 8 arrivals per hour	CPU and memory constraints All requests admitted
Main test	100 unused workloads Seeds 1000–1099	Separate testing from development
Inference	8 planned tests Paired 95% intervals, Holm correction	Control multiple-test false positives

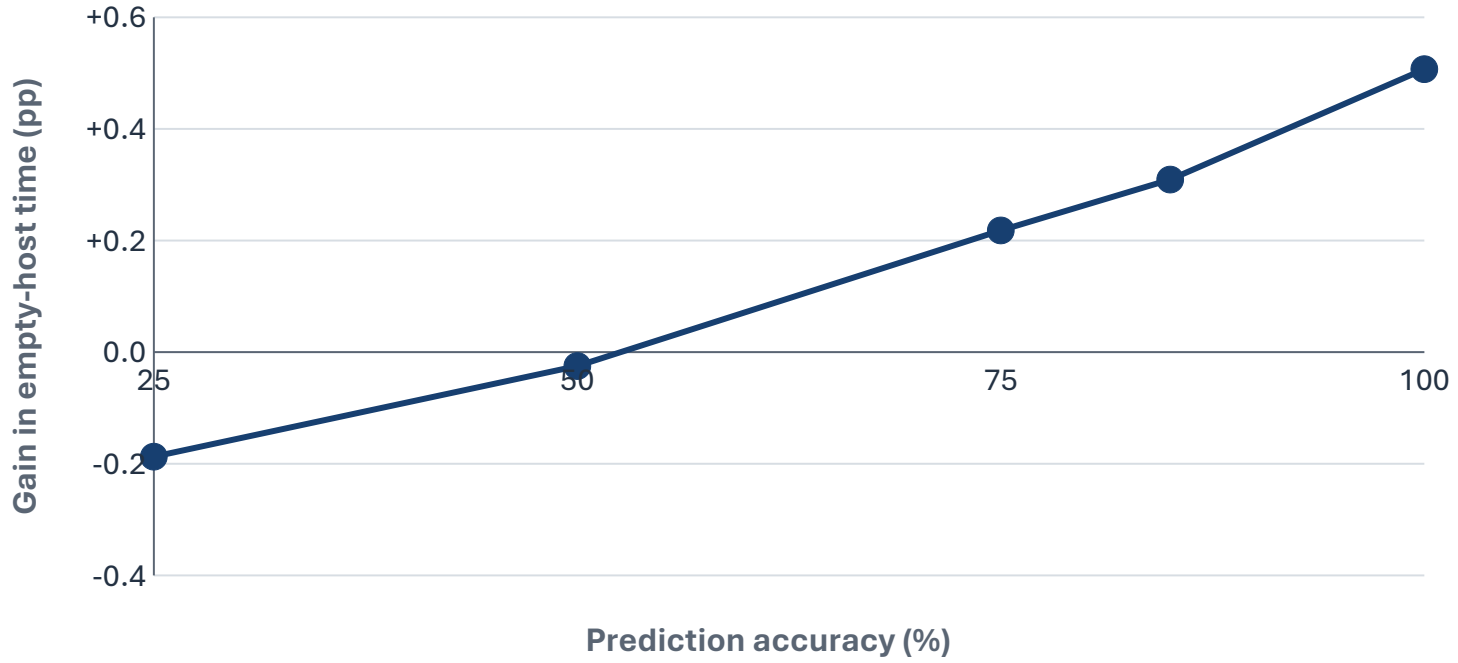
Metric: percentage of host-time completely empty, from first arrival to final exit.

Differences use percentage points (pp). +0.224 pp corresponds to about 0.40 extra empty hosts on average.

LAVA improves as prediction quality rises

Fresh-seed comparison: LAVA minus Best-Fit on the same 100 workloads.

Accuracy = probability of a near-correct estimate in this controlled test.



25% accuracy

−0.187 pp

95% CI [−0.240, −0.135]

100% accuracy

+0.507 pp

95% CI [+0.445, +0.569]

Poor estimates can make lifetime-aware placement worse. This motivates a way to handle unreliable predictions.

Confidence routing tries to protect LAVA

The first idea: send arrivals with unreliable lifetime estimates to Best-Fit.

Confidence = probability of the lifetime class containing the estimate. Threshold: 0.3.

Mechanism	VM-level CA	Host-level CA
Decision	Choose LAVA or Best-Fit for each arrival	Choose a LAVA or Best-Fit host pool
Host sharing	Both decisions use the same hosts	Keep host modes separate until empty
Intended benefit	Avoid trusting a weak prediction	Also avoid interference between the modes

A high probability for a lifetime class need not imply that choosing LAVA will improve packing.

The routing mechanism works, but the benefit fails

We diagnose the negative result rather than assume the confidence gate must help.

Observation	Evidence	Effect on scheduling
Score ranks profiles poorly	Default model: interactive 0.092, service 0.364	The most predictable profile gets the lowest score
Gate rarely rejects arrivals	Controlled noise: 99.96%–100% pass at $\tau = 0.3$	The gate does not filter the damaging errors
Isolation reduces packing choices	Default model: −0.992 pp during arrivals, −0.090 pp overall	A VM cannot freely use space in the other pool

Host isolation removes cross-mode placements, yet the default-model result is still below Best-Fit.

Prior fusion checks a prediction against usual behavior

Prior = the usual lifetime pattern for this kind of VM. Fusion combines it with a new estimate.

The prior asks: how plausible is this observation for this kind of VM?

Example	Prior mean	Noisy estimate	Fused estimate
Interactive VM **25% accuracy**	1.07 h	300 h	1.52 h
Service VM **50% accuracy**	85.2 h	2 h	27.0 h

An implausible observation gets less weight. A plausible observation can still update the prior.

We change the predictor while preserving the placement code.

Fusion uses the observation and continued survival

A VM still running supplies new evidence. A capped prediction also needs careful interpretation.

Step	What the predictor does
1 Start with usual behavior	Use the profile's lifetime distribution as the prior
2 Use the observation	Weight plausible lifetimes using the known noise model
3 Condition on uptime	Retain lifetimes consistent with the VM still running

Capped observation example

At 100% controlled accuracy: prediction capped at 336 h, VM already running for 300 h.

Original estimate: 36 h left. Fused estimate: 151 h left.

The registered improvement holds in 3 of 4 LAVA tests

Registered VM-CA-Bayes combines fusion with VM-level routing. Paired gains in pp, n = 100.

Accuracy	Gain over LAVA [95% CI]	Gain over Best-Fit [95% CI]
25%	+0.224 [+0.174, +0.275] Confirmed	+0.037 [-0.012, +0.086] Not confirmed
50%	+0.147 [+0.088, +0.205] Confirmed	+0.122 [+0.064, +0.180] Confirmed
75%	+0.054 [-0.001, +0.109] Not confirmed	+0.272 [+0.217, +0.327] Confirmed
85%	+0.088 [+0.037, +0.139] Confirmed	+0.398 [+0.338, +0.457] Confirmed

75% versus LAVA remains unconfirmed (Holm p = 0.11).

25% versus Best-Fit also remains unconfirmed. Intervals are unadjusted, decisions use Holm correction across 8 tests.

Decomposition points to the estimate as the main gain

Change one component at a time to separate the estimate from the confidence gate.

Paired comparison	At 25% accuracy	At 85% accuracy
Fuse instead of trusting the noisy value	+0.242 [+0.182, +0.303]	+0.105 [+0.054, +0.155]
Add the observation to the prior alone	+0.098 [+0.047, +0.149]	+0.457 [+0.398, +0.515]
Add a gate to the same fused estimate	-0.018 [-0.066, +0.030]	-0.016 [-0.066, +0.033]

Fusion improves on both ingredients in these examples. The added gate has intervals spanning zero.

Post-hoc analysis with unadjusted 95% intervals. It does not prove the gate has exactly zero effect.

A useful predictor still depends on its assumptions

Robustness changes one assumption at a time, using 50 fresh workloads per setting.

Change	Observed result	Interpretation
Milder incorrect predictions	Only 1–2 of 4 intervals above zero	Smaller errors are harder to distinguish
Prior lifetimes × 0.5	At 85%: −0.116 [−0.197, −0.034] pp vs LAVA	Underestimating long lifetimes can hurt
Prior lifetimes × 2	All four gains over LAVA have positive intervals	The direction of prior error matters
Twice the arrivals and requests	Gains over LAVA at 25–75%, 85% uncertain	Load changes the size of the benefit

The failing prior case is still +0.191 pp above Best-Fit. These robustness tests are exploratory and unadjusted.

Findings, contributions and limits

The evidence supports a predictor-level remedy within the reconstructed synthetic environment.

What we learned

Prediction quality matters

LAVA changes from −0.187 to +0.507 pp over Best-Fit.

Routing is insufficient here

Host isolation works, but confidence does not identify useful decisions.

The estimate is a promising target

The registered fused variant improves on LAVA in 3 of 4 tests.

What limits the claim

Synthetic workloads and low load

Generating prior and known error model

Local registration and exploratory robustness

Next: learn priors from history and test real workload changes.